

Federated Evolutionary Neural Architecture Search with FedAvg: Validation of Client Selection and Multiobjective Search

EvoFederated Research Group

Abstract

Federated neural architecture search (NAS) combines the cost of evolutionary search with the optimization and evaluation difficulties of non-IID client data. We examine a narrow formulation in which every candidate architecture is trained as one global model with FedAvg, and the search objectives are the mean and worst-client validation accuracy of that model. We compare NSGA-II, random search, NSGA-III, MOEA/D, a fixed MLP, and five partial-participation strategies based on random, similarity, and dissimilarity pair selection. The campaign uses Fashion-MNIST partitioned among eight clients with a Dirichlet concentration of $\alpha = 0.5$, and evaluates three paired seeds. With a population of eight architectures, seven generations, one federated round, and one local epoch, full participation requires 448 local trainings per search, whereas four-client participation requires 224. Across seeds, MOEA/D obtained the highest mean test accuracy among the matched-budget optimizers (0.758 ± 0.019), while NSGA-II had the highest mean worst-client accuracy (0.662 ± 0.052). Among partial-participation strategies, probabilistic dissimilarity obtained the highest mean test accuracy (0.732 ± 0.032), while deterministic dissimilarity obtained the highest mean worst-client accuracy (0.587 ± 0.080). Partial participation reduced search local trainings by 50%, but all partial methods had lower mean accuracy than the full-participation reference in paired comparisons. With three seeds, the experiment shows a measurable cost-quality trade-off but does not establish that one optimizer or client-selection heuristic is superior.

Keywords: federated learning, FedAvg, neural architecture search, non-IID data, evolutionary optimization, client selection

1 Introduction

Federated learning (FL) trains a shared model without centralizing the raw examples held by participating clients [1]. Statistical heterogeneity makes both optimization and evaluation difficult: an architecture that performs well on average may still fail on a particular client [2]. Neural architecture search (NAS) adds an outer optimization loop in which many candidate networks must be trained, making the choice of participating clients a direct computational concern.

This work evaluates a specific and deliberately narrow question: can evolutionary NAS be implemented as an outer search over architectures whose inner fitness is obtained from a single FedAvg-trained global model? The search uses validation data only. A candidate is locally trained from a common initialization, its client updates are aggregated by sample-weighted FedAvg, and the resulting global model is evaluated on every client’s validation split. The test split is held out until final re-training of selected architectures.

The experiment keeps three questions separate. A federated NAS pipeline must distinguish local data, global model state, validation, and test evaluation. Partial client participation may reduce search cost, but that reduction does not show that a particular client-selection heuristic improves

the selected architecture. We therefore report local-training counts and final client-level metrics, and do not treat architectures or generations as independent statistical replicates.

The paper contributes an evolutionary NAS protocol whose inner loop uses FedAvg, with mean and worst-client validation objectives defined on the resulting global model. It compares several evolutionary optimizers and client-participation strategies under a common protocol, records local-training and communication costs, and reports a three-seed Fashion-MNIST replication with paired descriptive uncertainty. The remaining sensitivity checks are stated explicitly rather than treated as completed results.

2 Related Work

Federated averaging provides a standard mechanism for aggregating locally trained models without collecting raw client data [1]. Recent surveys emphasize that non-IID data, device constraints, communication variability, and model heterogeneity affect federated optimization through different mechanisms [2, 19, 20]. These distinctions are important for experimental design: a client-selection rule based on update similarity is not, by itself, a solution to statistical, computational, and network heterogeneity simultaneously.

2.1 Federated neural architecture search

Federated neural architecture search (FedNAS) combines the outer search problem of NAS with the privacy and optimization constraints of federated learning. A dedicated survey organizes FedNAS by search paradigm, execution mode, and objective structure, covering evolutionary, reinforcement-learning, gradient-based, online, offline, single-objective, and multi-objective formulations [12]. We follow that framing but restrict the target to one deployable global model.

Several recent methods focus on reducing the cost of architecture search. Zhu and Jin propose real-time federated evolutionary NAS with double sampling of submodels and clients to reduce local computation and communication payload [13]. Pan et al. introduce multi-granular FedNAS, in which clients search at micro and macro architectural levels and a probabilistic aggregation mechanism combines local architecture information [14]. Yuan et al. propose a resource-aware FedNAS design that separates architecture search from model training and combines partial-client training, early candidate elimination, and dynamic round counts [15]. These approaches are closely related to the present cost-quality question, but they generally modify the NAS execution or search space to address resource limitations.

Another line of work targets heterogeneous or personalized architectures rather than a single global architecture. Finch clusters clients according to data distributions and performs hierarchical subnet search for client groups [16]. Peaches searches personalized architectures for edge workers whose local distributions differ substantially [17], while network-aware FedNAS incorporates network conditions into architecture selection [18]. These methods motivate explicit treatment of client and system heterogeneity, but their primary estimand is personalized or resource-adapted performance. Here, the experimental unit is one architecture, one global FedAvg trajectory, and a validation score defined by mean and minimum accuracy across clients. The result is a comparison protocol, not a new FedNAS optimizer.

2.2 Client selection and heterogeneous federated learning

Client selection methods seek to improve convergence, time-to-accuracy, or resource efficiency by choosing a subset of available clients. Oort combines data utility with execution speed and uses

guided participant selection to improve federated training and testing efficiency [21]. AUCTION learns a quality-aware selection policy with reinforcement learning from client status and federated feedback [22]. Other approaches construct participant utilities for heterogeneous data and devices [23], or formulate selection as a knapsack-constrained submodular optimization problem over systems and statistical heterogeneity [24]. These methods establish that the nominal number of participating clients is an incomplete proxy for computational cost: data volume, device speed, communication, and selection overhead can also matter.

Here we study a narrower intervention. Similarity and dissimilarity schedules are constructed from client update vectors obtained during a common probe, then compared at a fixed participation budget inside the same global FedAvg protocol. This design does not claim that update geometry dominates utility-based, speed-aware, or theoretically optimized selection. Instead, it isolates whether a simple update-based schedule changes the quality-cost trade-off when architecture search and final evaluation are held constant. Worst-client accuracy is used as a robustness objective; it is not interpreted as demographic fairness. FairFed illustrates this distinction by addressing group fairness through fairness-aware aggregation under heterogeneous populations [25].

Prior work shows that FedNAS cost, client selection, and heterogeneity are coupled. We therefore compare global FedAvg candidates using validation data, record mean and worst-client objectives together, and preserve local-training and communication costs separately. This design distinguishes a reduction in search cost from evidence that one selection heuristic produces a better global architecture.

3 Methods

3.1 Federated NAS formulation

Let $C_1, \dots, C_N$ be clients and let A be a decoded MLP architecture. For a candidate, FedAvg produces a global parameter vector $\theta^{(R)}$ after R rounds. The validation accuracy of that same global model on client i is $a_{\text{val}}(A, C_i)$. The two search objectives are

$$\bar{a}_{\text{val}}(A) = \frac{1}{N} \sum_{i=1}^N a_{\text{val}}(A, C_i), \quad a_{\text{val}}(A) = \min_i a_{\text{val}}(A, C_i). \quad (1)$$

The evolutionary implementation minimizes the negative of these quantities. The complete per-client validation vector is retained for each evaluated architecture.

Each client has disjoint local train, validation, and test subsets. During search, clients receive the current global state, train locally, and return derived model state and metrics. The server aggregates model parameters with weights proportional to local training-set size. Non-trainable normalization statistics use the same weighted average, whereas counter variables are aggregated by their maximum. Raw examples remain in the clients.

3.2 Architecture space and FedAvg

The genome is a normalized vector in $[0, 1]^{L_{\text{max}}+4}$. Its entries encode the number of hidden layers, one width value for each possible layer, the activation, dropout, and the BatchNorm flag. Decoding clips every entry to $[0, 1]$, rounds categorical entries, maps widths to $\{16, 32, 64\}$, maps dropout to $[0, 0.5]$, and ignores inactive width entries beyond the decoded depth. The campaign uses at most two hidden layers, ReLU or tanh activations, and the same architecture configuration for every method. The fixed baseline is a one-hidden-layer network with width 16, ReLU, zero dropout, and no BatchNorm; it is evaluated once and is not budget matched to the search methods.

For each architecture, the global model is initialized deterministically from a genome-dependent seed. In each federated round, the selected clients start from that same global state. If $U_i(\theta^{(r)})$ is the model returned by client i after local training and n_i is its number of training examples, the aggregation is

$$\theta^{(r+1)} = \sum_{i \in S_r} \frac{n_i}{\sum_{j \in S_r} n_j} U_i(\theta^{(r)}), \quad (2)$$

where S_r is the participant set for round r .

3.3 Algorithms

Algorithm 1 defines the inner fitness evaluation. The important distinction from independent-client evaluation is that all validation scores are obtained from the same aggregated global model $\theta^{(R)}$. The state passed between rounds includes trainable parameters and non-trainable BatchNorm buffers.

```

Algorithm 1: FEDAVG-FIT(A, schedule, D, E, eta)
Input: architecture A, round schedule S[1..R], client data D,
      local epochs E, learning rate eta
theta <- deterministic_initialize(A)
for r <- 1..R do
  updates <- empty map
  for client i in S[r] do
    model <- instantiate(A)
    load_global_state(model, theta)
    local_train(model, D[i].train, E, eta)
    updates[i] <- model_state(model)
  theta <- weighted_state_average(updates, weights=n_train[i])
scores <- [accuracy(evaluate(theta, D[i].val)) for i=1..N]
return (-mean(scores), -min(scores)), theta, scores

```

Figure 1: Algorithm 1. Pseudocode for the inner FedAvg fitness. Floating-point BatchNorm buffers are averaged with the same sample weights; integer counters use the maximum. No raw examples leave a client.

Client characterization is performed once at the start of each run, before any architecture search. A deterministic probe genome is decoded into an MLP and the same initial parameters are copied to every client. Each client trains that probe for one local epoch, and the parameter update $\Delta_i = \theta_i - \theta_0$ is retained. The probe is not a fitness model; it only supplies the update geometry used by the similarity-based and dissimilarity-based schedules. The resulting pairing is reused throughout the run. Random- k schedules are sampled separately for each architecture evaluation and round.

```

Algorithm 2: CHARACTERIZE-AND-BUILD-SCHEDULE(strategy, clients, k, tau)
probe_genome <- deterministic_random_genome()
theta0 <- initialize(probe_genome)
for client i do
    theta_i <- local_train(copy(theta0), client_i, one_epoch)
    delta[i] <- theta_i - theta0
S <- cosine_similarity(delta); D <- 1 - S
if strategy = full then
    return all clients for every round
if strategy = random-k then
    return seeded random samples of k clients per evaluation and round
if strategy = similarity then P <- max_weight_matching(S)
if strategy = dissimilarity then P <- max_weight_matching(D)
if strategy is probabilistic then
    while unmatched clients remain do
        choose an unmatched anchor i
        sample partner j with p(i,j) proportional to
            exp(S[i,j]/tau) or exp(D[i,j]/tau)
for round r do
    choose one representative from each pair using generation + r
    append any singleton clients
    schedule[r] <- sorted representatives
return schedule, S, D

```

Figure 2: Algorithm 2. Pseudocode for client characterization and participation construction. The probe and pairing are created once per run. For the completed campaign, all four pair representatives are used, so each search evaluation includes one representative from every pair; the realized τ and pair assignments are recorded for reproducibility.

The outer experiment has two branches. Phase 1 changes the architecture optimizer while keeping full participation fixed. Phase 2 keeps NSGA-II fixed and changes only the search-time participant schedule. In both phases, validation records are used for search and ranking; the held-out test split is used only after the search has selected candidates. NSGA-II and random search use the two aggregate validation objectives $(-\bar{a}_{\text{val}}, -a_{\text{val},\min})$, whereas NSGA-III and MOEA/D use the per-client vector $(1 - a_{\text{val},1}, \dots, 1 - a_{\text{val},N})$. The final ranking is nevertheless the same for all methods: validation mean first, then validation minimum.

```

Algorithm 3: FEDERATED-NAS(protocol, strategy, optimizer)
clients <- partition data into disjoint train/validation/test subsets
probe, S, D <- CHARACTERIZE-AND-BUILD-SCHEDULE(strategy, clients, k, tau)
population <- initialize_architecture_population()
for generation g <- 1..G do
  for architecture A in population do
    schedule <- BUILD-SCHEDULE(strategy, clients, k, tau)
    if optimizer in {NSGA-II, random} then
      F[A] <- (-mean_val[A], -min_val[A])
    else if optimizer in {NSGA-III, MOEA/D} then
      F[A] <- (1-acc_val[A,1], ..., 1-acc_val[A,N])
    F[A] <- FEDAVG-FIT(A, schedule, clients, E, eta)
  population <- optimizer_step(population, F, optimizer)
records <- deduplicate_by_genome(all_records)
ranked <- sort(records, mean_val descending, min_val descending)
top <- first three candidates in ranked
mark rank one as primary candidate
final_schedule <- full-client schedule unless
  final evaluation uses the search schedule
for architecture A in top do
  theta <- FEDAVG-FIT(A, final_schedule, clients, E, eta)
  test_scores[A] <- evaluate(theta, each_client_test_data)
return population_history, test_scores, cost

```

Figure 3: Algorithm 3. Pseudocode for the two experimental phases. Phase 1 changes the optimizer under full participation; Phase 2 changes the search-time schedule under NSGA-II. Three validation-ranked candidates are re-trained, and rank one supplies the primary run-level result. The default final step uses full participation and the held-out test split only after validation-based selection.

3.4 Experimental design

The study contains two related experiments. The first changes the architecture-search method while keeping full client participation fixed. The second keeps NSGA-II fixed and changes only the participant schedule used during the search. This separation makes the optimizer comparison and the participation comparison interpretable: the first varies the outer search procedure, whereas the second varies the clients used to train each candidate.

The common experimental unit is one architecture, one deterministic initialization, one FedAvg trajectory, and one evaluation of the resulting global model on all eight clients. Fashion-MNIST is partitioned by class with a Dirichlet concentration of $\alpha = 0.5$. Each client’s assigned training data is divided into local train and validation subsets; the official test set is partitioned independently and remains unused until final evaluation. The completed replication uses paired seeds 42, 123, and 999. All search conditions use one FedAvg round, one local epoch, Adam with learning rate 10^{-3} , batch size 128, and CPU execution.

3.4.1 Experiment 1: optimizer comparison

The optimizer experiment uses full participation for every architecture evaluation. It compares a fixed MLP, random search, NSGA-II, NSGA-III, and MOEA/D. The evolutionary methods use a nominal population of eight and seven generations, or 56 architecture evaluations. Random

Table 1: Optimizer conditions in the full-participation experiment. The objectives are written in minimization form.

Method	Search objective	Optimizer configuration	Evaluations
Fixed MLP	None	One hidden layer, width 16, ReLU, zero dropout, no BatchNorm	1
Random search	$(-\bar{a}_{\text{val}}, -a_{\text{val},\min})$	Independent uniform samples in the normalized genome space; no replacement step	56
NSGA-II	$(-\bar{a}_{\text{val}}, -a_{\text{val},\min})$	SBX crossover ($p = 0.9, \eta = 15$), polynomial mutation ($p = 0.2, \eta = 20$), duplicate elimination	56
NSGA-III	$(1 - a_{\text{val},1}, \dots, 1 - a_{\text{val},8})$	Das-Dennis reference directions, SBX ($p = 1, \eta = 30$), polynomial mutation ($\eta = 20$)	56
MOEA/D	$(1 - a_{\text{val},1}, \dots, 1 - a_{\text{val},8})$	Tchebycheff decomposition, eight reference directions, neighborhood mating probability 0.9, SBX and polynomial mutation with $\eta = 20$	56

Table 2: Participant schedules in the NSGA-II participation experiment. The probe is common to all conditions in a run and is not used as the candidate fitness model.

Strategy	Information used	Schedule construction during search	Local trainings
Full	None	All eight clients in every round	448
Random-4	None	Four seeded clients sampled without replacement for each architecture and round	224
Similarity	S	Maximum-weight matching; one alternating representative from each pair	224
Dissimilarity	$D = 1 - S$	Maximum-weight matching; one alternating representative from each pair	224
Probabilistic similarity	$S, \tau = 0.5$	Sequential softmax pairing; one alternating representative from each pair	224
Probabilistic dissimilarity	$D, \tau = 0.5$	Sequential softmax pairing; one alternating representative from each pair	224

search draws a new set of eight architectures at each generation and does not perform evolutionary replacement. The fixed MLP is evaluated once as a low-cost reference and is not treated as a budget-matched competitor. The optimizer-specific objectives and operators are given in Table 1. All methods use the same validation-based final ranking and the same full-participation test re-training.

3.4.2 Experiment 2: participation comparison

The participation experiment uses NSGA-II for every condition, the same 56 architecture evaluations, and a search budget of $k = 4$ clients per round for partial methods. A one-epoch probe is trained once per run, separately from the architecture evaluations. Its client update vectors define the cosine similarity matrix S and the dissimilarity matrix $D = 1 - S$. Deterministic methods use maximum-weight matching on S or D . Probabilistic methods choose an unmatched anchor and sample its partner with probability proportional to $\exp(S_{ij}/\tau)$ or $\exp(D_{ij}/\tau)$, with $\tau = 0.5$. The resulting pairs are fixed for the run, while representatives alternate with generation and round. With eight clients, four representatives are produced and all four are used, ensuring one participant from each pair in every search evaluation. The six schedules are summarized in Table 2.

3.5 Evaluation and cost

For every architecture, the resulting global model is evaluated separately on each client’s validation data. The recorded per-client outcomes are accuracy and macro-F1. The primary search objectives are the unweighted mean accuracy and the minimum client accuracy; mean and minimum macro-F1 are retained as secondary outcomes. Candidate records are deduplicated by decoded genome before selection, and the validation Pareto set and ranked candidates are retained for analysis.

Table 3: Common experimental protocol. All values are fixed across the optimizer and participation comparisons unless noted.

Component	Setting
Dataset	Fashion-MNIST
Clients and heterogeneity	$N = 8$, Dirichlet $\alpha = 0.5$
Splits	official train split partitioned into train/validation; official test partitioned independently and held out
Architecture space	MLP, $L_{\max} = 2$, widths $\{16, 32, 64\}$, ReLU/tanh, dropout $[0, 0.5]$, optional BatchNorm
Initialization	deterministic seed derived from the decoded genome
Evolution	population 8, 7 generations, seeds 42, 123, 999
Federated training	1 round, 1 local epoch, Adam, 10^{-3} learning rate, batch size 128
Probe	one local epoch per client, once per run, used only for update geometry
Participation budget	full or $k = 4$ clients per search round
Search objectives	validation mean/minimum accuracy, or one validation accuracy objective per client
Final selection	three validation-ranked candidates; rank one is the primary candidate
Evaluation	validation for search; test after full-participation re-training

After selection, the three highest-ranked validation candidates are re-trained from scratch with full participation in the completed replication and evaluated on the held-out test split. The rank-one candidate is marked as the primary candidate and supplies the run-level result used in the tables and figures. The other two final evaluations are exploratory and are included only in the final-evaluation cost.

Search cost is reported as local client trainings, probe trainings, federated rounds, communication messages, transmitted state bytes, and elapsed time. With 56 architecture evaluations, full participation requires $56 \times 8 = 448$ search local trainings, whereas four-client strategies require $56 \times 4 = 224$. Including the common eight-client probe and the three-candidate full-participation final evaluation, the corresponding totals are 480 and 256 local trainings. Communication counts one download and one upload of the model state for each local training. Elapsed time is the run-level wall-clock measurement and includes probe, search, final evaluation, and execution overhead. Hypervolume is recorded for each run as an internal optimizer diagnostic: NSGA-II uses two objectives, while NSGA-III and MOEA/D use one objective per client, so their hypervolumes are not pooled into a single cross-dimensional ranking.

3.6 Analysis plan

The primary descriptive outcomes are mean test accuracy and minimum test accuracy across clients. Macro-F1 is reported as a secondary outcome. Search efficiency is quantified by local-training count, transmitted state bytes, and elapsed time. We summarize the three run-level values and compute paired differences by seed; architectures, clients, and evolutionary generations are not treated as independent replicates. Figures 1–3, Figures 5 and 7, and the hypervolume panel in Figure 9 use two-sided t intervals around the mean across three seeds. The client-trajectory panel in Figure 9 reports means without uncertainty bands. All intervals are descriptive rather than confirmatory, and the appropriate replication unit remains the paired seed and partition.

4 Results

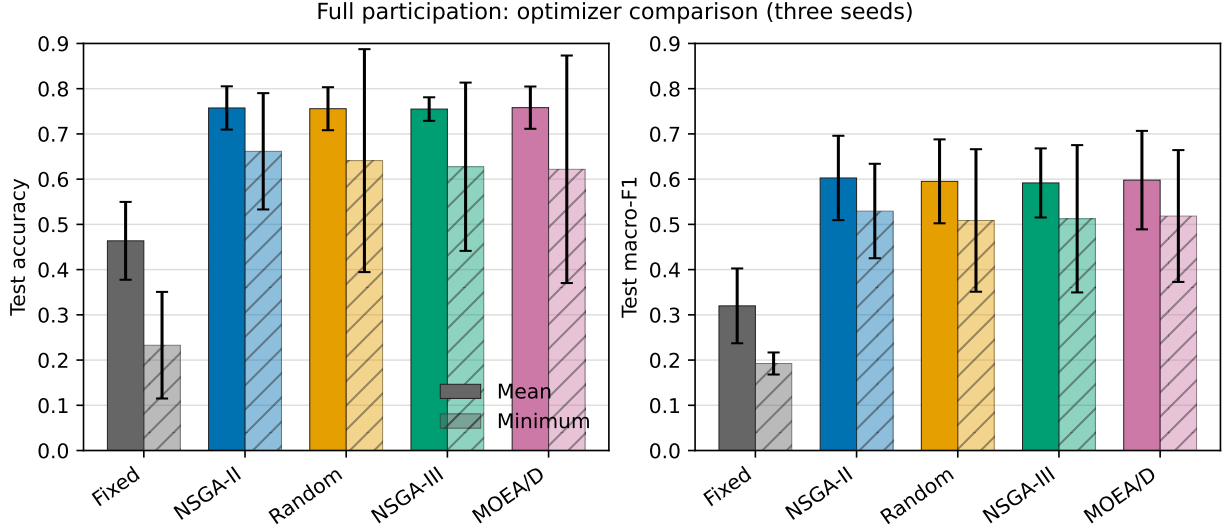


Figure 4: Phase 1 optimizer experiment under full participation. The five conditions are evaluated over three paired seeds; bars show final test mean and minimum client accuracy and macro-F1, the hatched bars denote minimum-client metrics, and error bars are 95% descriptive t intervals across seeds.

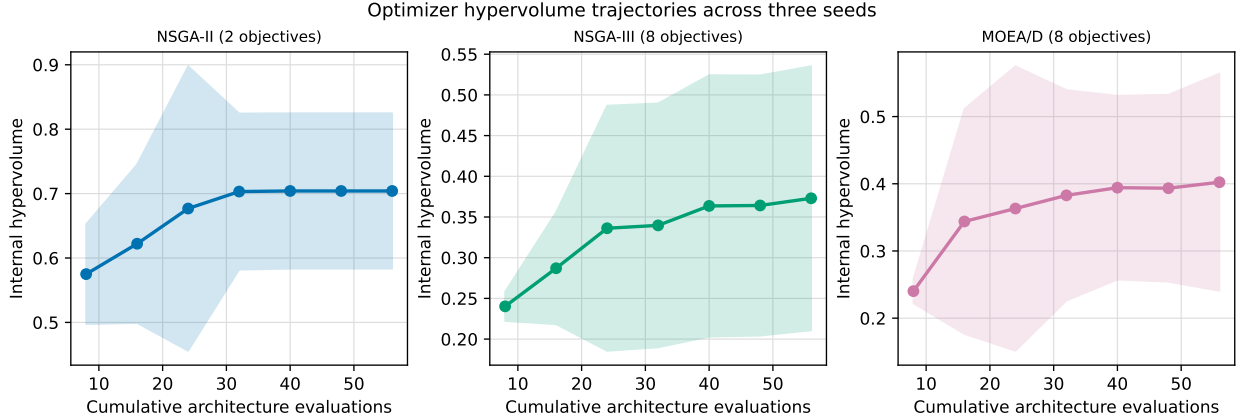


Figure 5: Internal hypervolume trajectories for the three multiobjective optimizers in the full-participation experiment. Each panel uses the optimizer’s own objective space: two aggregate validation objectives for NSGA-II and eight per-client validation objectives for NSGA-III and MOEA/D. Lines and shaded regions show means and 95% descriptive t intervals across the three seeds; values are compared only within a panel.

4.1 Optimizer comparison

Table 4 and Figure 4 report the Phase 1 full-participation comparison. Each bar summarizes the primary rank-one candidate from one run, not all architectures evaluated during the search. MOEA/D produced the highest mean test accuracy, 0.758 (SD 0.019), and the highest mean test macro-F1, 0.598 (SD 0.044). NSGA-II had the highest mean minimum accuracy, 0.662 (SD 0.052),

Table 4: Full-participation optimizer comparison on Fashion-MNIST. Entries are mean $\pm$ SD across three seeds; search uses validation and reported metrics are final held-out test values.

Method	Arch. evals	Local trainings	Mean acc.	Min acc.	Mean F1	Min F1
Fixed MLP	1	8	0.464 $\pm$ 0.035	0.233 $\pm$ 0.047	0.320 $\pm$ 0.033	0.192 $\pm$ 0.010
NSGA-II	56	448	0.757 $\pm$ 0.019	0.662 $\pm$ 0.052	0.603 $\pm$ 0.038	0.530 $\pm$ 0.042
Random search	56	448	0.756 $\pm$ 0.019	0.641 $\pm$ 0.099	0.595 $\pm$ 0.037	0.509 $\pm$ 0.063
NSGA-III	56	448	0.755 $\pm$ 0.010	0.627 $\pm$ 0.075	0.592 $\pm$ 0.031	0.512 $\pm$ 0.066
MOEA/D	56	448	0.758 $\pm$ 0.019	0.622 $\pm$ 0.101	0.598 $\pm$ 0.044	0.518 $\pm$ 0.059

Table 5: NSGA-II participation comparison. Entries are mean $\pm$ SD across three seeds; the full row is the full-participation NSGA-II reference from Table 4.

Strategy	Arch. evals	Local trainings	Mean acc.	Min acc.	Mean F1	Min F1
Full	56	448	0.757 $\pm$ 0.019	0.662 $\pm$ 0.052	0.603 $\pm$ 0.038	0.530 $\pm$ 0.042
Random-4	56	224	0.722 $\pm$ 0.003	0.580 $\pm$ 0.099	0.565 $\pm$ 0.023	0.485 $\pm$ 0.056
Similarity	56	224	0.725 $\pm$ 0.031	0.581 $\pm$ 0.128	0.566 $\pm$ 0.047	0.473 $\pm$ 0.080
Dissimilarity	56	224	0.730 $\pm$ 0.032	0.587 $\pm$ 0.080	0.567 $\pm$ 0.050	0.507 $\pm$ 0.062
Probabilistic similarity	56	224	0.728 $\pm$ 0.030	0.586 $\pm$ 0.078	0.559 $\pm$ 0.036	0.494 $\pm$ 0.040
Probabilistic dissimilarity	56	224	0.732 $\pm$ 0.032	0.582 $\pm$ 0.082	0.565 $\pm$ 0.035	0.501 $\pm$ 0.036

while MOEA/D had the highest mean minimum macro-F1, 0.518 (SD 0.059). Paired differences relative to NSGA-II were small relative to their seed-to-seed variation, so these results do not establish optimizer superiority.

The fixed MLP is substantially below all search methods in mean accuracy and macro-F1, but it is not a matched-budget competitor. Among the four matched-budget search procedures, the mean accuracy range is narrow (0.755–0.758), and the uncertainty intervals overlap. MOEA/D has the highest mean accuracy, NSGA-II has the highest mean minimum accuracy, and MOEA/D has the highest mean minimum macro-F1. This crossing of objectives does not identify a dominant optimizer.

Figure 4 makes the distinction between average and worst-client performance explicit. The ranking changes by metric, and the intervals reflect variation across the three paired seeds. The fixed baseline is included for context, but its smaller search budget prevents a fair optimizer comparison. Figure 5 adds the corresponding search trajectories for NSGA-II, NSGA-III, and MOEA/D. Because the methods optimize different numbers of objectives, the panels use separate scales and are not a cross-method hypervolume ranking.

4.2 Participation comparison

Table 5 and Figure 6 report the Phase 2 comparison under a fixed NSGA-II search. The full row is the full-participation reference from Phase 1, matched by seed; it is not a separate Phase 2 rerun. Partial methods reduce search local trainings from 448 to 224, a 50% reduction, while total local trainings fall from 480 to 256, a 46.7% reduction, because probe and final evaluation costs remain. Relative to the full reference, mean test accuracy changes by -0.035 for random-4, -0.032 for deterministic similarity, -0.028 for deterministic dissimilarity, -0.030 for probabilistic similarity, and -0.026 for probabilistic dissimilarity. Probabilistic dissimilarity has the highest mean accuracy among partial methods, while deterministic dissimilarity has the highest mean minimum accuracy; no partial strategy consistently dominates the others on all quality metrics.

The observed reduction in search cost is clear, but the quality comparison is not decisive. All partial methods have lower mean accuracy than the full reference on average, although probabilistic

dissimilarity has the highest mean accuracy and deterministic dissimilarity has the highest mean minimum accuracy among partial methods. Figure 7 shows the corresponding two-objective search trajectories; these values are comparable across participation schedules because the optimizer and objective space are fixed. The paired intervals are wide for worst-client accuracy, especially with only three seeds, so these rankings should remain descriptive.

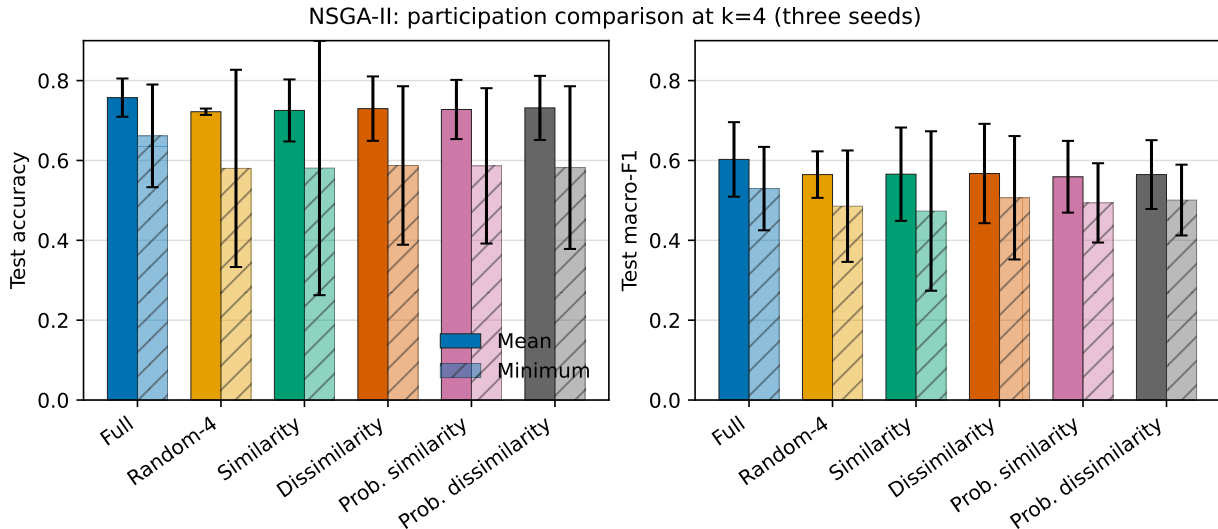


Figure 6: Phase 2 participation experiment under a fixed NSGA-II search. The full row is the Phase 1 full-participation reference; the other bars use four clients per search round. Final test mean and minimum client metrics are shown separately, with hatched bars denoting the minimum.

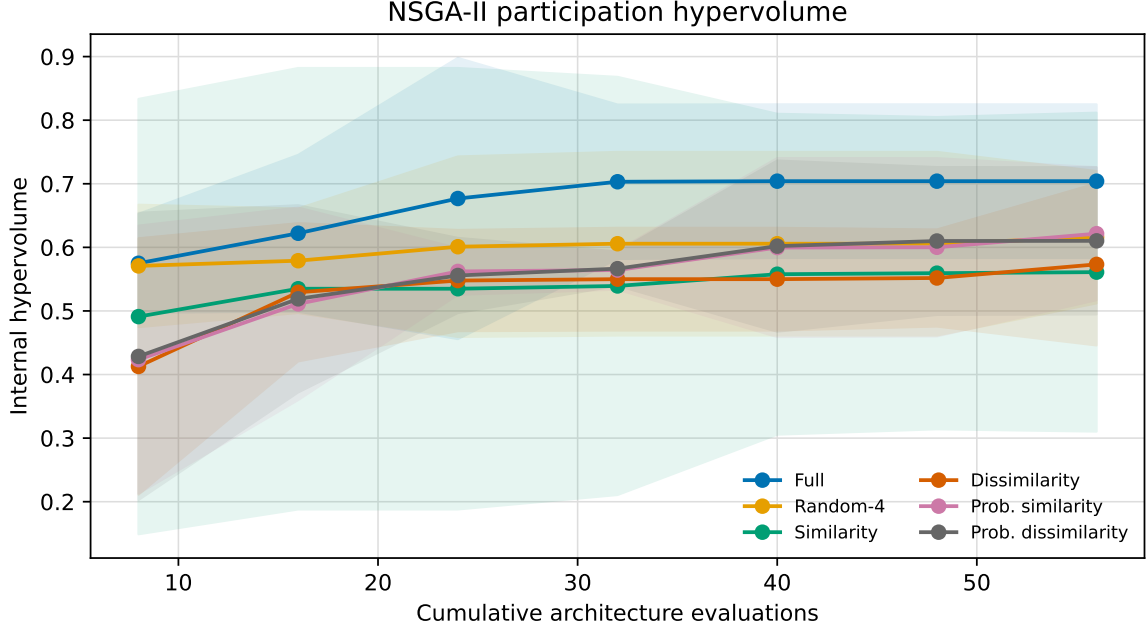


Figure 7: Internal hypervolume trajectories for the six NSGA-II participation conditions. All curves use the same two aggregate validation objectives and reference point, so values are comparable across schedules. Lines and shaded regions show means and 95% descriptive t intervals across the three seeds.

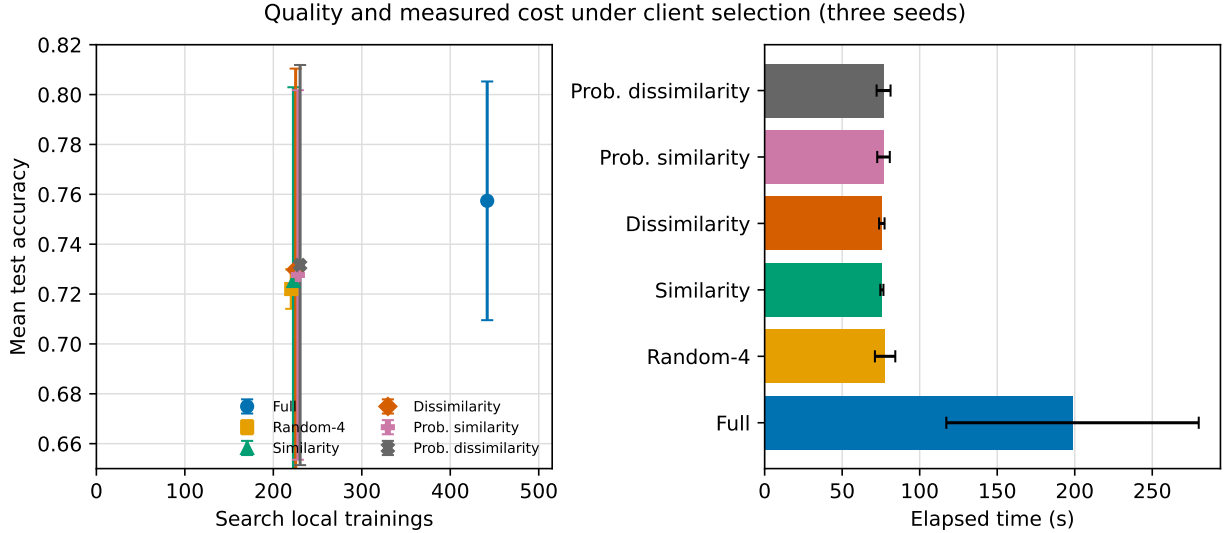


Figure 8: Quality and cost from the same Phase 2 participation experiment. The left panel relates search local trainings to final test mean accuracy; the right panel reports run-level elapsed CPU time, including probe, search, final evaluation, and execution overhead. Error bars are 95% descriptive t intervals; the exact search costs are 448 for full participation and 224 for each partial strategy.

The cost-quality plot in Figure 8 shows that local-training count and elapsed time are not interchangeable. All partial strategies use 224 search local trainings, but measured elapsed times vary across seeds because client schedules, architecture trajectories, and CPU execution overhead

differ. The final evaluation is full participation for every method and therefore contributes a common 24 local trainings per run. Probe, search, final evaluation, communication, and elapsed time remain separate quantities instead of being reduced to one nominal participation cost.

4.3 Validation boundary and reproducibility checks

All search outcomes were computed from validation data, and the held-out test data were used only after architecture selection. A separate leakage check confirmed that test evaluation did not occur during validation-based search. For each run, we recorded the partition assignments and class counts, probe outcomes, participant schedules, validation and final Pareto sets, per-client final metrics, and the component-wise cost ledger. Machine-readable run summaries and paired descriptive estimates are provided with the reproducibility materials.

4.4 Search dynamics and client heterogeneity

Figure 9 is a search-history view of the full-participation NSGA-II condition from Phase 1, not a fourth experiment. The hypervolume panel uses the internal two-objective validation space $(-\bar{a}_{\text{val}}, -a_{\text{val},\min})$ and the fixed reference point $(0.1, 0.1)$, and shows the mean recorded value after each generation with a descriptive interval. The client panel selects the candidate with the highest validation mean within each generation and displays the mean validation accuracy on every client. It has no uncertainty bands because it is a client-level trajectory summary, and it does not use held-out test data.

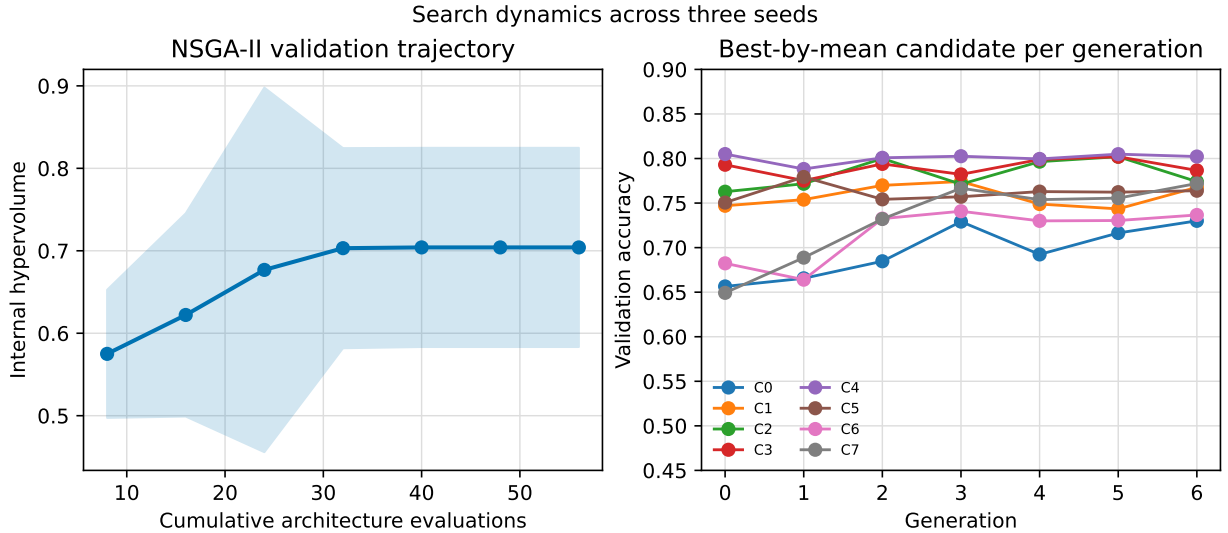


Figure 9: Search history for the full-participation NSGA-II condition across three seeds. Left: mean internal two-objective validation hypervolume versus cumulative architecture evaluations, with a descriptive t interval. Right: mean validation accuracy for each client after selecting the candidate with the highest validation mean in each generation; no test data or uncertainty band is used in this panel.

The client trajectories illustrate why a mean-only analysis is insufficient. Client accuracies occupy different ranges and the candidate selected by validation mean does not necessarily maximize every client score. The minimum-client objective records this conflict during search, while the final test table evaluates whether the selected global model transfers to the held-out split.

5 Discussion

The experiment defines one unit of evaluation as one architecture, one deterministic initialization, one FedAvg trajectory, and one mean/worst-client validation score. Under the selected configuration, four-client participation reduces search local trainings by 50% relative to full participation, and total local trainings by 46.7% after including the common probe and final evaluation. This is a systems result; it does not show that a selection heuristic preserves global quality.

The three-seed results do not support the hypothesis that divergence-based selection is superior to random selection. Probabilistic dissimilarity has a mean test accuracy of 0.732, deterministic dissimilarity 0.730, probabilistic similarity 0.728, deterministic similarity 0.725, and random-4 0.722, while the paired differences relative to full participation are negative for every partial strategy. Deterministic dissimilarity has the highest mean minimum accuracy among the partial methods, but its seed-level variation remains substantial. These differences could arise from the partition, initialization, optimizer trajectory, or candidate selection, and should not be interpreted as causal effects from three paired seeds.

The optimizer comparison is also descriptive. NSGA-II, random search, NSGA-III, and MOEA/D use the same architecture-evaluation count and full participation, but differ in objective dimension and internal evolutionary behavior. The many-objective methods optimize one objective per client, so their internal Pareto hypervolumes are not directly comparable with the two-objective NSGA-II hypervolume; this is why Figure 5 keeps the trajectories in separate panels. In contrast, Figure 7 compares participation schedules within the same two-objective NSGA-II space.

The results also show why figures should be read with the accompanying tables. Figure 6 summarizes final client aggregates, Figure 8 exposes the cost trade-off, and Figure 9 shows the validation trajectory. The intervals are descriptive across three seeds, not evidence of a powered hypothesis test; the source data and transformations are recorded in the accompanying provenance table.

6 Limitations and next experiment

This is a three-seed replication, not a definitive statistical campaign. It uses one dataset, eight clients, one federated round, one local epoch, and a reduced MLP search space. Three paired seeds permit descriptive intervals but are too few for strong claims about optimizer or client-selection superiority. The fixed MLP baseline is not budget-matched to the evolutionary optimizers. Because the final evaluation uses full participation for every condition, the participation comparison isolates search cost rather than end-to-end deployment cost. The characterization probe and elapsed CPU time are also separate components of total cost, and CPU time varies across runs.

The next campaign should test heterogeneity sensitivity by repeating the validated subset at $\alpha = 0.1$ and, if resources permit, $\alpha = 10$. It should also verify that tau changes the realized participant schedules before interpreting tau or probe-length ablations. A larger population, more rounds, and a second dataset are useful extensions, but they should follow the current matched-budget replication rather than replace it.

7 Conclusion

We evaluated a federated evolutionary NAS protocol in which candidates are trained as global FedAvg models and selected using validation mean and worst-client accuracy. Across three paired Fashion-MNIST seeds, MOEA/D had the highest mean test accuracy among matched-budget

optimizers, while NSGA-II and MOEA/D performed best on different worst-client summaries. Four-client participation reduced search local trainings by 50%, but every partial strategy had lower mean accuracy than the full-participation reference on average. The results separate the measured efficiency of the search from evidence about the quality of the resulting global model.

Data, Code, and Transparency

Code and machine-readable replication summaries are available in the accompanying repository or supplementary archive. Raw dataset files are not redistributed, and figure provenance is provided alongside the figures. AI tools assisted with code and manuscript preparation under the documented workflow [11]; the authors remain responsible for verifying the scientific claims, citations, and final version.

References

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of AISTATS*, 2017.
- [2] P. Kairouz et al., “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, nos. 1–2, 2021.
- [3] B. Zoph and Q. V. Le, “Neural architecture search with reinforcement learning,” in *Proceedings of ICLR*, 2017.
- [4] H. Pham, M. Guan, B. Zoph, Q. V. Le, and J. Dean, “Efficient neural architecture search via parameter sharing,” in *Proceedings of ICML*, 2018.
- [5] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, “Regularized evolution for image classifier architecture search,” in *Proceedings of AAAI*, 2019.
- [6] T. Elsken, J. H. Metzen, and F. Hutter, “Neural architecture search: A survey,” *Journal of Machine Learning Research*, vol. 20, no. 55, 2019.
- [7] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, 2002.
- [8] K. Deb and H. Jain, “An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I,” *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, 2014.
- [9] Q. Zhang and H. Li, “MOEA/D: A multiobjective optimization algorithm based on decomposition,” *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, 2007.
- [10] E. Zitzler, L. Thiele, L. Laumanns, C. M. Fonseca, and V. da Fonseca, “Performance assessment of multiobjective optimizers: An analysis and review,” *IEEE Transactions on Evolutionary Computation*, vol. 7, no. 2, 2003.
- [11] T. Kassis, V. Agarwal, Y. He, D. Patel, and A. M. Brueckner, “Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents,” arXiv:2609.00065, 2026. doi:10.48550/arXiv.2609.00065.

- [12] H. Zhu, H. Zhang, and Y. Jin, “From federated learning to federated neural architecture search: A survey,” *Complex & Intelligent Systems*, vol. 7, no. 2, pp. 639–657, 2021. doi:10.1007/s40747-020-00247-z.
- [13] H. Zhu and Y. Jin, “Real-time federated evolutionary neural architecture search,” *IEEE Transactions on Evolutionary Computation*, vol. 26, no. 2, pp. 364–378, 2022. doi:10.1109/TEVC.2021.3099448.
- [14] Z. Pan, H. Li, W. Tang, J. Li, Y. He, and Z. Liu, “Privacy-preserving multi-granular federated neural architecture search: A general framework,” *IEEE Transactions on Knowledge and Data Engineering*, 2021. doi:10.1109/TKDE.2021.3116248.
- [15] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang, “Resource-aware federated neural architecture search over heterogeneous mobile devices,” *IEEE Transactions on Big Data*, pp. 1–11, 2024. doi:10.1109/TBDATA.2022.3227403.
- [16] J. Liu, J. Yan, H. Xu, Z. Wang, J. Huang, and Y. Xu, “Finch: Enhancing federated learning with hierarchical neural architecture search,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 6012–6026, 2024. doi:10.1109/TMC.2023.3315451.
- [17] J. Yan, J. Liu, H. Xu, Z. Wang, and C. Qiao, “Peaches: Personalized federated learning with neural architecture search in edge computing,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 11, pp. 10296–10312, 2024. doi:10.1109/TMC.2024.3373506.
- [18] G. Ocal and A. Ozgovde, “Network-aware federated neural architecture search,” *Future Generation Computer Systems*, vol. 162, art. no. 107475, 2025. doi:10.1016/j.future.2024.07.053.
- [19] Z. Lu, H. Pan, Y. Dai, X. Si, and Y. Zhang, “Federated learning with non-IID data: A survey,” *IEEE Internet of Things Journal*, vol. 11, no. 11, pp. 19188–19209, 2024. doi:10.1109/JIOT.2024.3376548.
- [20] M. Ye, X. Fang, B. Du, P. C. Yuen, and D. Tao, “Heterogeneous federated learning: State-of-the-art and research challenges,” *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–44, 2024. doi:10.1145/3625558.
- [21] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, “Oort: Efficient federated learning via guided participant selection,” in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pp. 19–35, USENIX Association, 2021.
- [22] Y. Deng, F. Lyu, J. Ren, H. Wu, Y. Zhou, Y. Zhang, and X. Shen, “AUCTION: Automated and quality-aware client selection framework for efficient federated learning,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1996–2009, 2022. doi:10.1109/TPDS.2021.3134647.
- [23] J. Zhao, X. Chang, Y. Feng, C. H. Liu, and N. Liu, “Participant selection for federated learning with heterogeneous data in intelligent transport system,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 1, pp. 1106–1115, 2023. doi:10.1109/TITS.2022.3149753.
- [24] J. Zhang, J. Wang, Y. Li, F. Xin, F. Dong, J. Luo, and Z. Wu, “Addressing heterogeneity in federated learning with client selection via submodular optimization,” *ACM Transactions on Sensor Networks*, vol. 20, no. 2, pp. 1–32, 2024. doi:10.1145/3638052.

- [25] Y. H. Ezzeldin, S. Yan, C. He, E. Ferrara, and A. S. Avestimehr, “FairFed: Enabling group fairness in federated learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 6, pp. 7494–7502, 2023. doi:10.1609/aaai.v37i6.25911.